

Actividad 11

Laboratorio: File Path Traversal, Validation of Start of Path

PortSwigger Web Security Academy

Elaborado por: Juan Pablo Casanova De La Madrid

Herramienta utilizada: Burp Suite Community Edition

1. Nombre del laboratorio

File path traversal, validation of start of path

Laboratorio perteneciente a PortSwigger Web Security Academy, dentro de la categoría File path traversal.

2. Nivel

Practitioner.

A diferencia de los laboratorios anteriores, en este caso la aplicación no recibe únicamente el nombre del archivo, sino la ruta completa del recurso como parámetro. La mitigación implementada consiste en validar que dicha ruta comience con el directorio esperado, sin verificar el resto de su contenido, lo que exige una técnica de bypass distinta a las vistas previamente (ruta absoluta, secuencias anidadas y doble codificación URL).

3. Tipo de explotación

Path Traversal / Directory Traversal mediante bypass de validación de prefijo (CWE-22: Improper Limitation of a Pathname to a Restricted Directory).

Es una vulnerabilidad del lado del servidor que permite a un atacante manipular la ruta de un archivo solicitado para acceder a archivos fuera del directorio raíz previsto por la aplicación. En este laboratorio, la explotación aprovecha que el control de seguridad solo comprueba que la ruta recibida comience con el directorio permitido (/var/www/images/), sin validar ni bloquear las secuencias de traversal que puedan añadirse a continuación de ese prefijo.

4. Objetivo del laboratorio

El laboratorio contiene una vulnerabilidad de path traversal en la funcionalidad que muestra las imágenes de los productos de la tienda. La aplicación transmite la ruta completa del archivo a través de un parámetro de la petición y valida que la ruta proporcionada comience con la carpeta esperada.

Objetivo: explotar dicha vulnerabilidad conservando el prefijo de directorio validado por la aplicación y añadiendo a continuación una secuencia de traversal, con el fin de recuperar el contenido del archivo del sistema /etc/passwd y dar por resuelto el laboratorio.

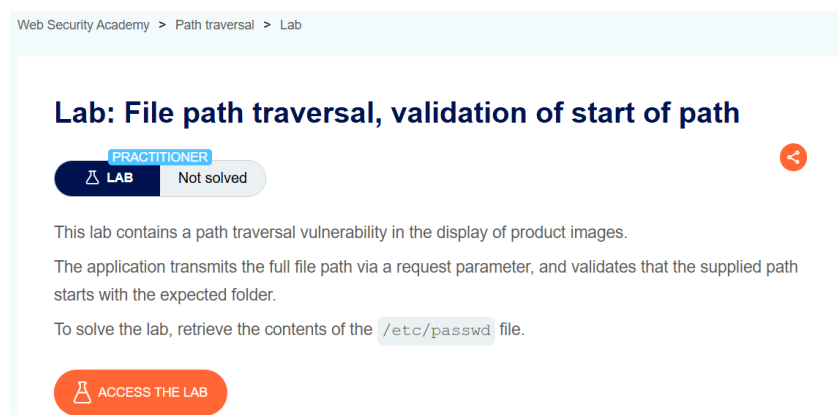


Figura 1. Descripción y objetivo del laboratorio en PortSwigger Web Security Academy (nivel Practitioner, estado Not solved).

5. Explicación técnica de la vulnerabilidad

A diferencia de los laboratorios previos, en este caso la aplicación no recibe un simple nombre de archivo, sino la ruta completa del recurso a través del parámetro filename, por ejemplo:

```
GET /image?filename=/var/www/images/52.jpg
```

Como mecanismo de protección, la aplicación valida que el valor recibido en filename comience exactamente con la ruta del directorio de imágenes permitido (/var/www/images/). Si la ruta no inicia con ese prefijo, la petición es rechazada.

El problema es que esta validación solo verifica el inicio de la cadena, sin comprobar que el resto de la ruta permanezca dentro de ese mismo directorio. Esto permite conservar el prefijo válido y, a continuación, agregar una secuencia de retroceso de directorios (../) que el sistema operativo interpretará al resolver la ruta completa, saliendo del directorio de imágenes aunque la cadena, en su inicio, cumpla con el control de seguridad.

Como resultado, una ruta como /var/www/images/../../../../etc/passwd supera la validación de prefijo (ya que comienza con /var/www/images/), pero al ser resuelta por el sistema de archivos, cada "../" retrocede un nivel de directorio hasta salir por completo de la carpeta de imágenes y llegar a la raíz, desde donde se referencia /etc/passwd. Esta vulnerabilidad se clasifica dentro de CWE-22 y ejemplifica los riesgos de validar únicamente el inicio de una ruta sin normalizarla ni verificar su destino final.

6. Procedimiento paso a paso

1. Se inicia Burp Suite Community Edition, seleccionando "Temporary project in memory" y la configuración por defecto ("Use Burp defaults").

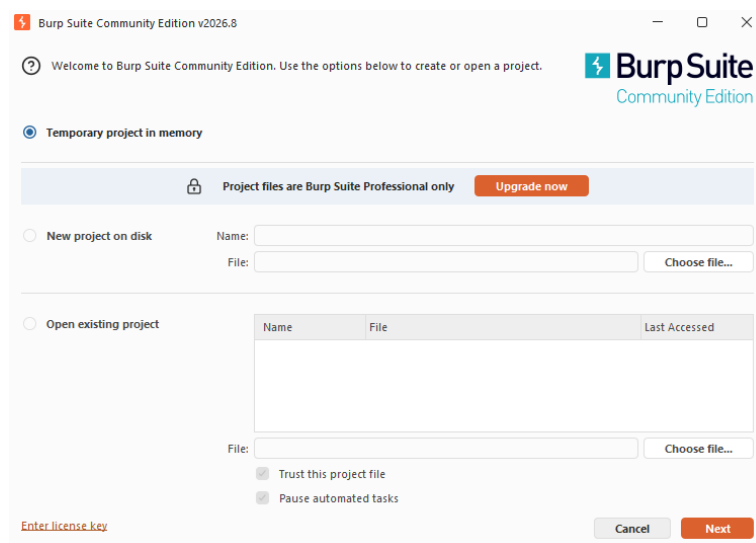


Figura 2. Creación de un proyecto temporal en memoria.

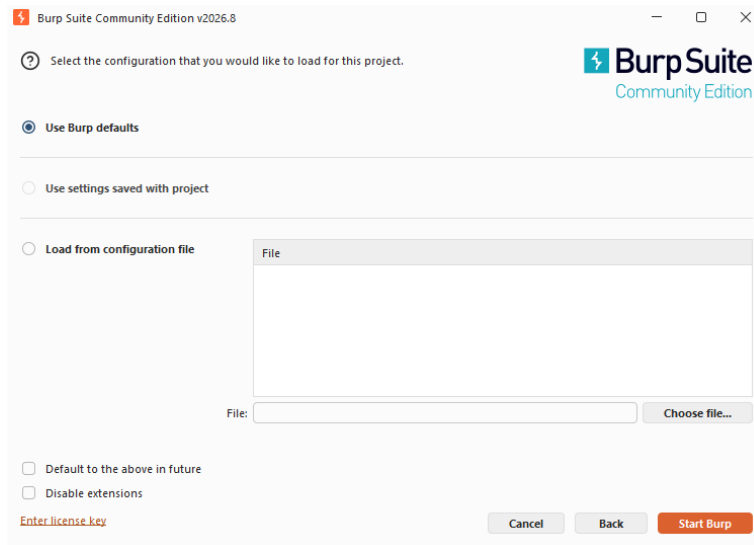


Figura 3. Selección de la configuración por defecto de Burp.

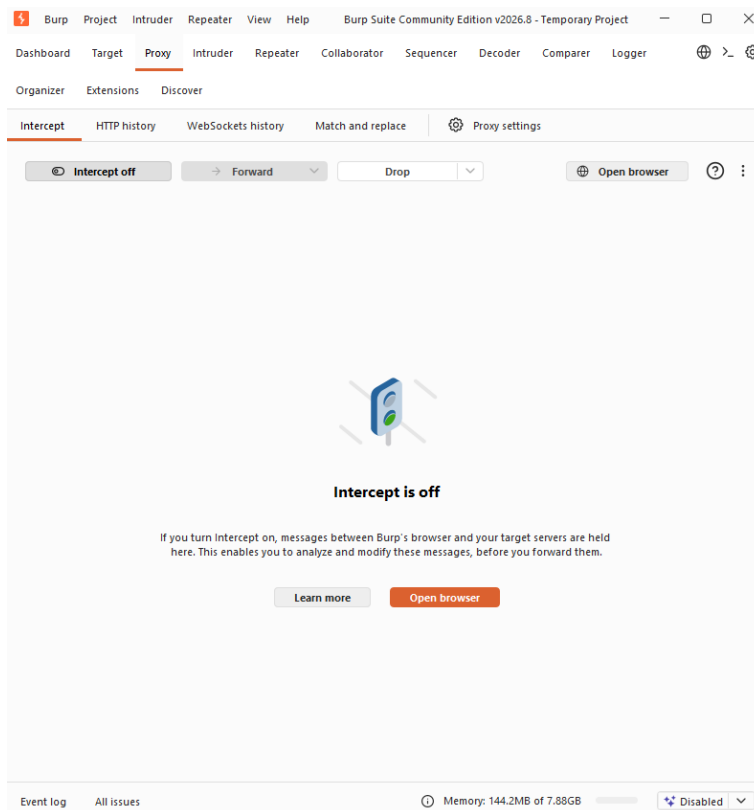


Figura 4. Burp Suite iniciado, pestaña Proxy con Intercept inicialmente apagado.

2. Se accede al laboratorio desde la Web Security Academy de PortSwigger, con el tráfico pasando a través del navegador integrado de Burp.

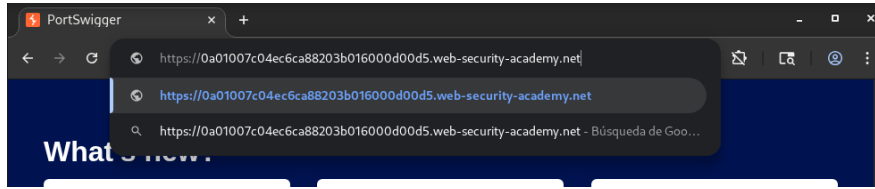


Figura 5. Navegación hacia la URL del laboratorio a través del navegador de Burp.

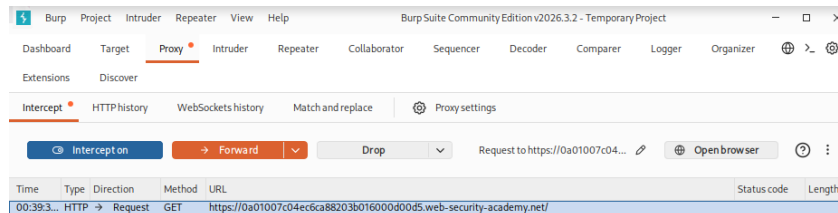


Figura 6. Petición GET inicial capturada en el historial del Proxy al abrir el laboratorio.

3. Con el Intercept activo, se navega por la tienda; Burp captura las peticiones GET que la página realiza para cargar cada imagen de producto, confirmando que el parámetro filename transmite la ruta completa (/var/www/images/NN.jpg) y no solo el nombre del archivo.

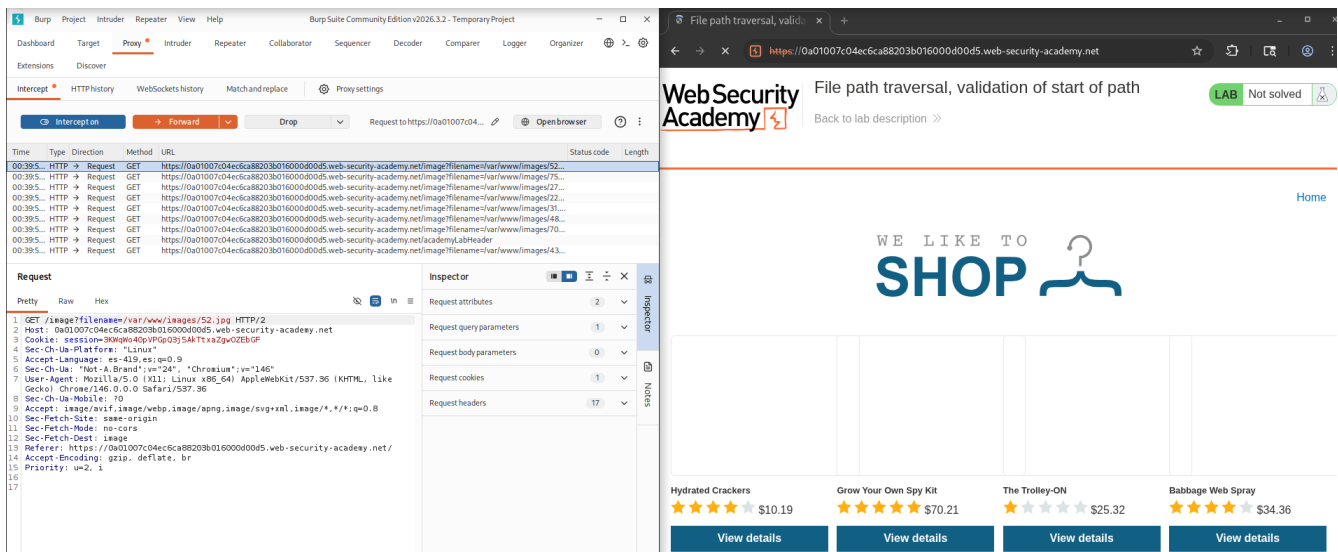


Figura 7. Petición interceptada GET /image?filename=/var/www/images/52.jpg junto con el resto de las peticiones de imágenes en el historial del Proxy.

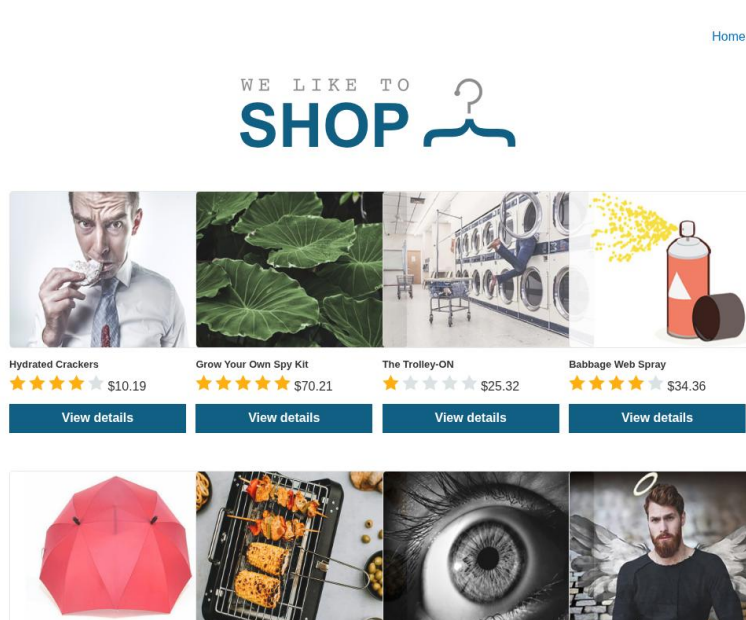


Figura 8. Vista de la tienda mostrando los productos cuyas imágenes se cargan mediante el parámetro filename.

- Se da clic en "View details" de uno de los productos, con el fin de capturar el método GET correspondiente y poder modificar sus parámetros.

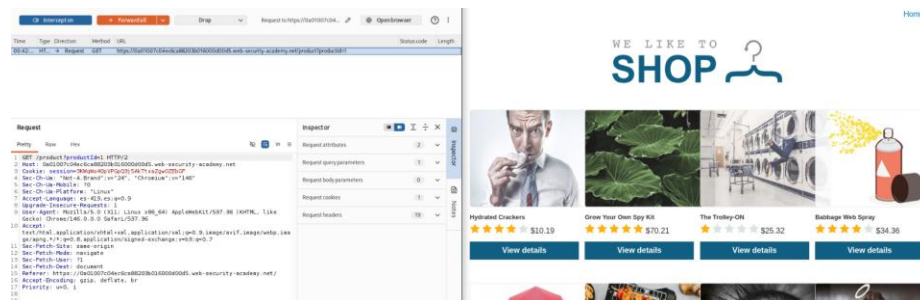


Figura 9. Petición GET /product?productId=1 interceptada tras dar clic en "View details".

- Se identifica la petición GET /image?filename=/var/www/images/52.jpg y, mediante clic derecho, se envía al módulo Repeater ("Send to Repeater") para poder manipular la ruta.

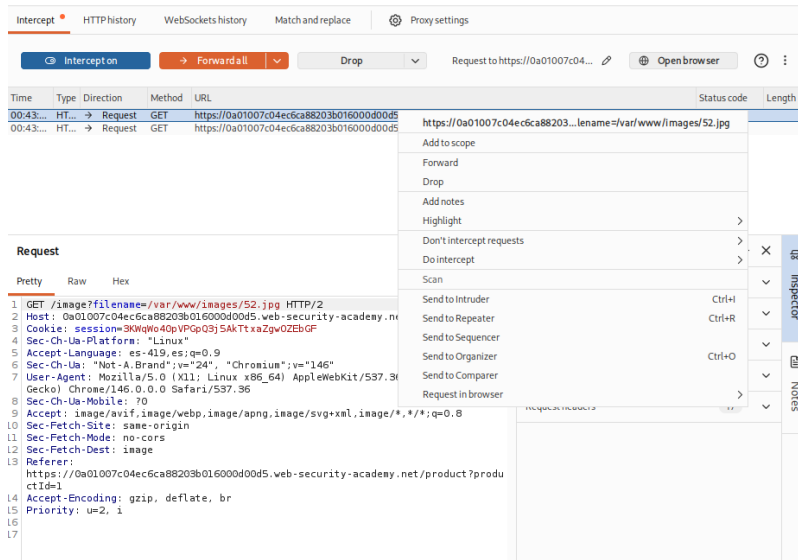


Figura 10. Menú contextual: envío de la petición GET /image?filename=/var/www/images/52.jpg a Repeater.

- En Repeater, dentro de GET /image?filename=/var/www/images/52.jpg, se conserva el prefijo /var/www/images/ (para superar la validación de inicio de ruta) y se sustituye 52.jpg por ../../../../etc/passwd, de modo que el sistema de archivos resuelva la ruta completa fuera del directorio de imágenes.

GET /image?filename=/var/www/images/../../../../../etc/passwd HTTP/2

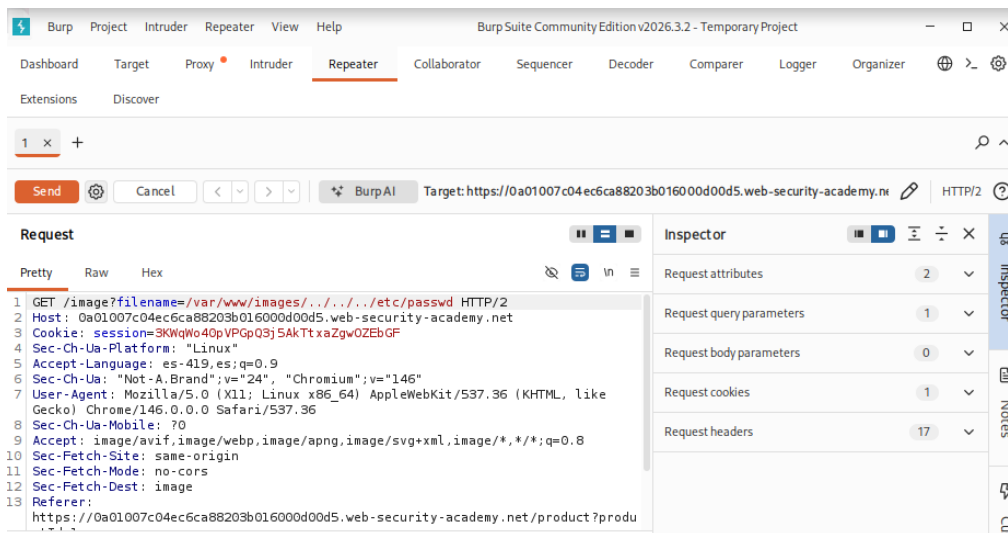


Figura 11. Payload con validación de prefijo evadida, inyectado en el parámetro filename.

- Se hace clic en Send y se analiza la respuesta del servidor.
- El servidor responde con código HTTP/2 200 OK y devuelve en el cuerpo de la respuesta el contenido íntegro del archivo /etc/passwd, confirmando la explotación exitosa de la vulnerabilidad.

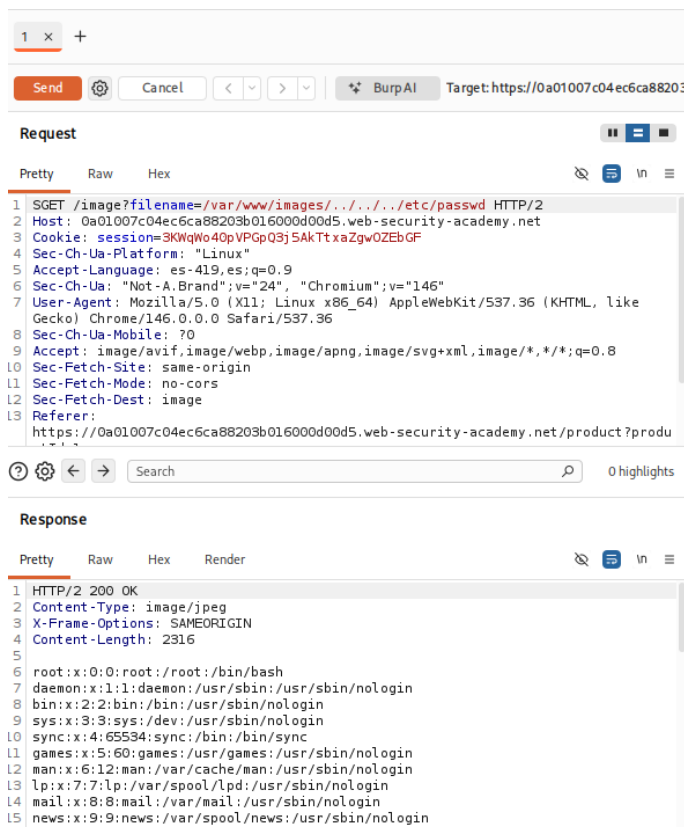


Figura 12. Petición con el payload y respuesta vulnerable (200 OK) mostrando el contenido de /etc/passwd.

9. Finalmente, se regresa a la página del laboratorio en PortSwigger, la cual actualiza automáticamente su estado a "Solved", confirmando que el objetivo de la actividad fue completado.

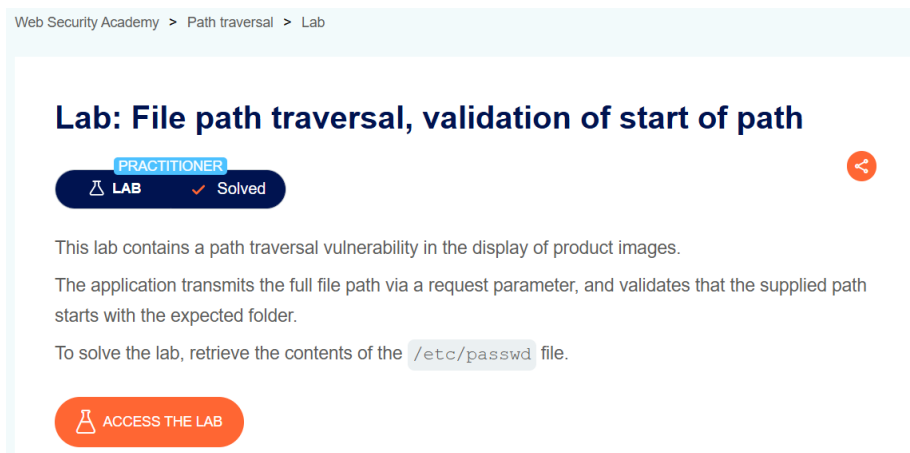


Figura 13. Confirmación de laboratorio resuelto ("Solved").

7. Explicación del payload

GET /image?filename=/var/www/images/../../../../etc/passwd HTTP/2

El payload se construye a partir de dos componentes que trabajan en conjunto:

- **/var/www/images/**: es el prefijo exacto que la aplicación exige al inicio del parámetro filename. Al conservarlo intacto, la validación de seguridad (que solo comprueba el comienzo de la cadena) se supera sin ser detectada, ya que, en apariencia, la ruta solicitada pertenece al directorio autorizado.
- **../../../../etc/passwd**: una vez superada la validación, esta secuencia se concatena directamente después del prefijo permitido. Cuando el sistema operativo resuelve la ruta completa, cada "../" retrocede un nivel de directorio a partir de /var/www/images/, por lo que basta con encadenar tres para salir de esa carpeta y llegar a la raíz del sistema de archivos, desde donde se referencia etc/passwd.

En síntesis, el payload no intenta ocultar ni disfrazar la secuencia de traversal, sino que aprovecha que el control de seguridad solo audita el prefijo de la ruta y confía en que el resto de la cadena permanecerá dentro del directorio permitido, sin verificarlo realmente tras la resolución de la ruta.

8. Mitigación recomendada

Para prevenir vulnerabilidades de path traversal como la explotada en este laboratorio, se recomienda aplicar las siguientes medidas, de forma combinada:

- No validar únicamente el prefijo de la ruta recibida; resolver la ruta de forma canónica (canonicalización) y verificar que el resultado final completo permanezca dentro del directorio base autorizado, incluyendo cualquier secuencia de traversal que pudiera contener.
- Evitar recibir rutas completas del lado del cliente; en su lugar, transmitir únicamente un identificador o nombre de archivo simple, y que sea el servidor quien construya internamente la ruta completa hacia el directorio permitido.
- Validar el parámetro filename contra una lista blanca (whitelist) de nombres de archivo o identificadores permitidos, en lugar de depender de la validación de un prefijo de ruta.
- Rechazar cualquier valor de entrada que contenga secuencias de retroceso de directorio ("../", "..\\") o sus variantes codificadas, independientemente de si la ruta comienza o no con el directorio esperado.
- Ejecutar el proceso de la aplicación con el mínimo privilegio necesario, restringiendo los permisos de lectura del usuario del servicio web únicamente a los directorios estrictamente necesarios.
- Reforzar la defensa en profundidad con un WAF (Web Application Firewall) que detecte patrones de path traversal en cualquier posición de la ruta, y aplicar monitoreo/alertas sobre accesos a archivos fuera de lo esperado.